PowerGraph-LLM: Novel Power Grid Graph Embedding and Optimization with Large Language Models

Fabien BERNIER[1], Jun CAO*[2], Maxime CORDY[1], Salah GHAMIZI[2,1]
[1]*SnT, University of Luxembourg, Luxembourg*
[2]*Luxembourg Institute of Health (LIH), Luxembourg*
Email: fabien.bernier@uni.lu, jun.cao@list.lu, maxime.cordy@uni.lu, salah.ghamizi@lih.lu

arXiv:2501.07639v3 [cs.AI] 27 Jan 2026

***Abstract*—Efficiently solving Optimal Power Flow (OPF) problems in power systems is crucial for operational planning and grid management. There is a growing need for scalable algorithms capable of handling the increasing variability, constraints, and uncertainties in modern power networks while providing accurate and fast solutions. To address this, machine learning techniques, particularly Graph Neural Networks (GNNs) have emerged as promising approaches. This letter introduces PowerGraph-LLM, the first framework explicitly designed for solving OPF problems using Large Language Models (LLMs). The proposed approach combines graph and tabular representations of power grids to effectively query LLMs, capturing the complex relationships and constraints in power systems. A new implementation of in-context learning and fine-tuning protocols for LLMs is introduced, tailored specifically for the OPF problem. PowerGraph-LLM demonstrates reliable performances using off-the-shelf LLM. Our study reveals the impact of LLM architecture, size, and fine-tuning and demonstrates our framework's ability to handle realistic grid components and constraints.**

***Index Terms*—Graph Embedding, LLM, Low-Rank Adaptation (LORA)**

I. Introduction

Resolving Alternating Current Optimal Power Flow (AC OPF) problems is a routine task in the operational planning of Power Systems (PS). Nevertheless, with the increasing variability, constraints, and uncertainties in today's power networks, solving problems accurately presents a significant challenge for power system engineers. Numerous strategies for addressing AC OPF challenges incorporate Machine Learning (ML) elements to mitigate the computational challenges associated with traditional optimization-based OPF approaches.

Graph Neural Networks (GNN) have recently demonstrated solid performance for various tasks in the power system. In particular, Liu et al. [1] proposed a new topology-informed GNN approach by combining grid topology and physical constraints. Ghamizi et al. [2] demonstrated that a heterogeneous graph representation combined with physical constraint losses leads to the best performances for PF and OPF tasks. Recent work such as SafePowerGraph [3] provided a standardized representation and benchmark of GNN for PF and OPF problems and identified the best architectures and design choice to solve these problems with GNN.

While these models achieve remarkable performance, they require expensive data curation — by collecting large training datasets with OPF solvers — and costly training for specific power grid sizes. Recent progress in foundation models, including LLMs (e.g. ChatGPT, LLaMa), has significantly reshaped the field of machine learning. Such models can indeed generalize to new tasks without expensive training, given the right indications. LLMs have also recently been explored to solve PS-related tasks. [4] proposed an LLM agent in the Deep Reinforcement Learning (DRL) training loop, directly allowing to model linguistic objectives and constraints in the OPF problem. The optimization is, however, run using traditional methods (solvers and DRL). A foundation model, developed in [5], can iteratively solve the OPF optimization problem by minimizing the cost function. The optimization only supports the toy example of a simple economic dispatch problem of units and does not fully optimize and predict the OPF variables (generation and bus variables) nor consider real world grid components (multiple loads, line operating limits). To the best of our knowledge, PowerGraph-LLM is the first embedding and optimization framework for OPF that supports realistic grid components and constraints.

This letter presents three main contributions. We first propose novel power grid embedding for OPF to query LLMs with graph and tabular representations, followed by the development of tailored in-context learning and fine-tuning protocols for these models. We finally include an empirical study on how the architecture, size, and fine-tuning of LLMs affect their performance in solving OPF problems.

II. PowerGraph-LLM Framework

We present in Fig. 1 our PowerGraph-LLM framework. The first three steps consist in building the correct message format to query an LLM for OPF using appropriate table or graph embeddings. The following steps are either implemented using open-source LLMs (Llama), or remote API (OpenAI).

a) Power Grid Embedding (blue steps 1, 2, and 3 in Fig. 1): We extend SafePowerGraph [3] to build the appropriate embedding for LLM. Starting from an initial grid descriptor (in MatPower, PandaPower or OpenDSS formats), we generate a Pytorch HeteroData with each component as a distinct subgraph (Step (1)). Each node can be of type: bus, load, generator, slack, or line and is associated with its distinct set of features X_b, X_l, X_g, X_s, X_e respectively. In order to create

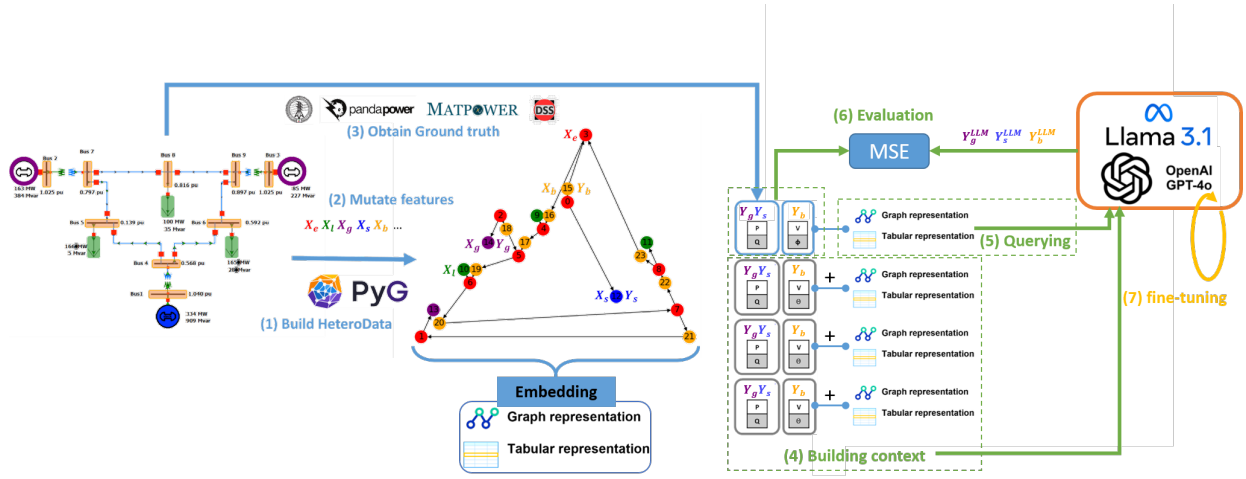

Fig. 1: The PowerGraph-LLM framework: We generate the power grid embedding in steps 1, 2, 3 where we obtain a description of the topology of the grid, the features of its components (X_b, X_l, X_g, X_s, X_e respectively for bus, loads, generators, slack, and lines), and the OPF solution by a solver (Y_g, Y_s, Y_b). We generate thousands of power grid embeddings and we split them into a context and queries. The context (step 4) is the pairs (power grid description + OPF solutions) that will serve for the LLM model as examples, and the query (step 5) is the grid to which we expect the model to perdict an OPF solution.

new grids, these features are mutated in Step (2) depending on their types. For example, the mutation for loads of the active and reactive power is based on the real profile.

For each mutated grid, Step (3) runs a simulation and solver to derive the PF or OPF optimization's ground truth. This result includes the active and reactive power for every generator and the slack nodes (Y_g, Y_s), as well as the voltage magnitude and angle for each bus node (Y_b).

These features are then prepared for in-context inference in two formats. Our approach can generate LLM messages formatted as *graph representations*, including nodes features and edges connections. We can also generate LLM messages that only contain the features formatted in a tabular format, and we refer to this as the *tabular representation*. Given the context size of an LLM (i.e., how many tokens can be used as input), we can encode more examples in a table representation than in a graph representation.

b) LLM Inference (green steps 4, 5, and 6 in Fig. 1): LLMs predict the next token in a sequence by using contextual information from previously seen words to generate coherent text, facilitated by a chat interface allowing dialogue between a *user* and an *assistant*. The dialogue can be conditioned by a *system prompt*.

A prompt consists in two parts, the *context*, which consists in pairs of examples and expected answers and the *query*, which is the actual input we expect the LLM to predict. In Step (4), the context is pairs of inputs/outputs. The grids (in tabular or graph embedding from the previous step) are the input examples, and the OPF solutions ((Y_g, Y_s, Y_b)) are the output examples. After providing a few examples, we query the LLM in step (5) with the actual grid to predict its OPF solution. The query only consists in the grid embedding either in tabular or graph representation without the OPF solutions.

The whole discussion consisting in the system prompt, the context and the query is processed in text format by the LLM. GPT and Llama models [6], two state-of-the-art LLMs families we consider in this paper, share a common high-level architecture — consisting in breaking text into tokens, converting them to embeddings, and processing them through transformer blocks to capture data patterns [7].

c) LLM fine-tuning with LoRA: The Low-Rank Adaptation (LoRA) [8] method allows fine-tuning LLMs efficiently by introducing low-rank matrices that capture task-specific adaptations while keeping the main model weights unchanged. Formally, instead of updating all the weights, LoRA modifies a small set by weight update ΔW as:

$$\Delta W = \frac{\alpha}{r} A \cdot B \tag{1}$$

where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, with $r \ll d, k$ and α being a chosen scaling factor. Only A and B are trained, reducing computation and memory needs, allowing for efficient adaptation to new tasks with few data and lower expenses while maintaining the LLMs' pre-existing capabilities.

We run the fine-tuning process in step (7) in yellow in Fig. 1. We consider each input example as a separate training sample and minimize the learning loss to its associated OPF solution.

III. Empirical Study

A. Experimental Protocol

a) In-context inference: To investigate the capability of LLMs in generalizing from examples presented within the context window, we conduct an assessment using in-context inference. Four models were tested: OpenAI's *gpt-4o-mini*, OpenAI's *gpt-4o*, *Llama-3.1-8B-Instruct*, and *Llama-3.1-70B-Instruct* [6], the latter being respectively renamed *llama-8b* and *llama-70b* for the sake of brevity. The inference for *llama* models is performed using *ollama* v0.5.0, with its default setup, including a context window of 4096 tokens.

Sequences provided for in-context inference are made as follows: after an initial system prompt, a total of 65 pairs of example requests and solutions are provided using the JSON

TABLE I: Errors in OPF estimation after fine-tuning GPT4o-mini and Llama-8b.

Case	LLM	Before fine-tuning				After fine-tuning			
		MSE_{GEN}	MSE_{SLACK}	MSE_{BUS}	VALID	MSE_{GEN}	MSE_{SLACK}	MSE_{BUS}	VALID
9-bus	GPT4o-mini (graph)	2.06×10^{-1}	1.78×10^{-1}	8.06×10^{-4}	97.7%	1.87×10^{-2}	1.07×10^{-2}	2.69×10^{-4}	93.1%
	Llama-8b (graph)	8.46×10^{6}	2.21×10^{3}	7.47×10^{-2}	11.5%	6.42×10^{-3}	6.18×10^{-2}	6.29×10^{-4}	99.7%
	Llama-8b (table)	3.17	1.96	1.92×10^{-3}	32.1%	5.44×10^{-2}	5.23×10^{-2}	1.85×10^{-3}	98.4%
30-bus	Llama-8b (graph)	-	-	-	0%	1.81×10^{-2}	1.68×10^{-2}	5.13×10^{-4}	89.5%

format. A context of 65 examples maximizes the utilization of context windows across all models. An additional, 66th example is used to evaluate the model's generalization abilities, with its response benchmarked against an expected solution. The overall sequence constructed is illustrated in Table II.

TABLE II: Sequence schema provided for LLM inference

system: You are a power grid operator running an Optimal Power Flow simulation, and you need to return a JSON-formatted response based on the provided input JSON. The input is the description of the components of the grid, including the buses, generators, loads, lines, and external grid. The output is the solution to the optimal power flow problem. You will get a few examples of Input and Output JSON. You need to return the correct Output for the last given Input.
user: Example Input JSON: <embedding input #1>
assistant: Example Output JSON: <OPF solution #1>
. . .
user: Query Input JSON: <embedding input #66>

If no JSON object can be read from the LLM's response, or if the values returned by the LLM are invalid (missing or invalid values), the output is deemed *INVALID*. This evaluation process was repeated 1,000 times, resulting in the generation of a total of 65,000 pairs for context and 1,000 pairs for evaluation across the assessments.

We run these inference tasks on one NVIDIA RTX 8000 48GB for Llama 8B (two for Llama 70B), using Q4_K_M quantization to optimize performance and resources.

b) Fine-tuning: Subsequently, we apply fine-tuning to the *Llama-3.1-8B-Instruct* model, as shown on Figure 1, using the 65,000 pairs of context developed earlier. Each fine-tuning sample includes the system prompt, the specific OPF problem being addressed, and its corresponding solution. We follow a similar protocol to fine-tune OpenAI's *gpt-4o-mini* with API.

The fine-tuning process of the *Llama* model was conducted on an NVIDIA RTX 8000 48GB, using a LoRA configuration. The LoRA setup was defined with a rank of 8 ($r = 8$) and a scaling factor of 16 ($\alpha = 16$). Both fine-tuning and inference of the fine-tuned model were performed using the Hugging Face library.

c) Evaluation: In all experiments, we evaluate the mean squared error (MSE) for the active and reactive powers of the generators and the slack bus, as well as the voltage magnitude and phase angle of the buses. Each test grid is an IEEE 9-bus grid where the loads are mutated following a uniform distribution (+/- 20% variations). We also report the percentage of test grids where the output of the LLM was *invalid*, i.e. no JSON could be parsed from the output. This typically happens when the assistant tries to explain how to solve the problem instead of providing the solution itself. Reported values include MSE for generators (GEN), slack buses (SLACK), and buses (BUS) values in the grid.

B. Effectiveness of pre-trained LLMs

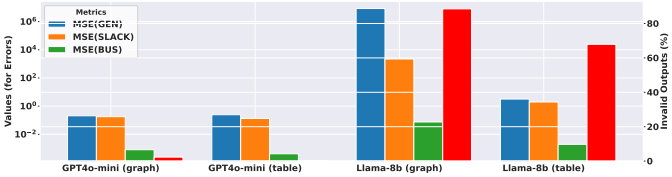

Fig. 2: Errors in OPF estimations for *gpt-4o-mini* and *llama-8b*. The red bars represent the proportion of invalid inputs.

a) ChatGPT (proprietary) vs. Llama (open-source): The results, presented in Figure 2, clearly show an important discrepancy between *gpt-4o-mini* and *llama-8b*. While *gpt-4o-mini* consistently showcases GEN and SLACK MSE values below 0.5 and BUS MSE below 10^{-3} for both graph and table formats with minimal invalid outputs, *llama-8b* exhibits higher MSEs by multiple magnitudes (min. 100 times more) for all criteria and a large majority of invalid outputs.

b) Graph vs. tabular representations: The usage of the tabular representation here results in lower errors than graphs for both models, although it is less pronounced for the GEN and SLACK with *gpt-4o-mini*. Table representations also lead to less invalid outputs by 23%, supporting their efficiency.

C. Impact of the size of the model

As it is the case for natural language tasks [6], bigger models result in better performance (Figure 3). This improvement is especially prominent for *llama-70b*, getting closer to *gpt-4o-mini*'s performance in terms of GEN and SLACK MSE.

The MSE of BUS decreases when increasing the size of the LLM using table representation but increases significantly with graph representation. Our results confirm that larger LLMs lead to better performance for given representations, and both the size and the representation parameters should be considered together in future assessments.

D. Impact of fine-tuning

We report in Table I the impact of fine-tuning LLMs to solve the OPF problem. The percentage of invalid outputs of the LLM decreases to less than 2% for the Llama models and marginally increases for the GPT4o model. The error across all the components decreases, in particular for the Llama models. They become as effective as the proprietary model.

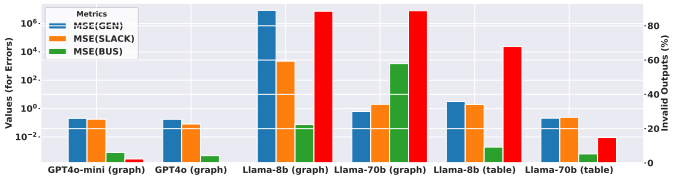

Fig. 3: Errors in OPF estimations for graph representations using bigger *gpt-4o* and *llama* models. The red bars represent the proportion of invalid inputs.

Contrary to the earlier vanilla models, graph representation is more effective than tabular to query LLMs after fine-tuning.

In Fig. 4, we compare the predicted active power of the generator and the ground truth. After fine-tuning, the model achieves faithful results within large ranges of perturbations. Similarly, Fig. 5 shows that the fine-tuned LLM leads to closer results while achieving only rare violations (Bus 2).

We focus on the graph representation to evaluate the generalization of larger grids. Our results on the 30-bus grid in Table I demonstrate that off-shelf LLMs completely fail to generate valid OPF solutions, but that LLMs can be fine-tuned to achieve competitive results with graph embedding.

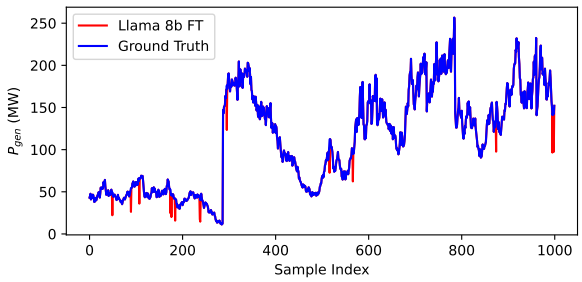

Fig. 4: Predicted and real value (Gen 1) with Llama-8b.

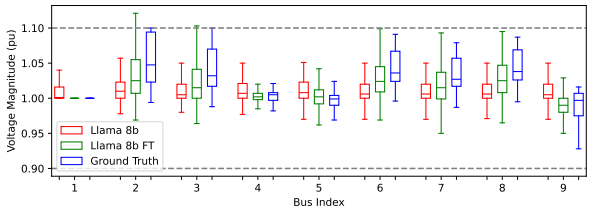

Fig. 5: Predicted and real voltage of buses with Llama-8b.

Conclusion

Our letter introduces PowerGraph-LLM, a novel framework for solving OPF problems using LLMs. We propose a new power grid embedding combining graph and tabular representations, LLM fine-tuning protocols for OPF, and an empirical analysis of LLM performance. Our results show that larger models perform better, with fine-tuning significantly improving accuracy and reducing invalid outputs. Graph representations become more effective than tabular ones after fine-tuning. These findings highlight the potential of LLMs in power system optimization and open new avenues for more efficient and accurate OPF solutions for complex power grids.

Acknowledgements

This work was partially funded by FNR CORE project LEAP (17042283) and by Creos Luxembourg S.A.
Dr. Ghamizi is supported by the Luxembourg National Research Fund (FNR) CORE C24/IS/18942843.

References

[1] S. Liu, C. Wu, and H. Zhu, "Topology-Aware Graph Neural Networks for Learning Feasible and Adaptive AC-OPF Solutions," *IEEE Transactions on Power Systems*, vol. 38, no. 6, pp. 5660–5670, Nov. 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9992121/

[2] S. Ghamizi, A. Ma, J. Cao, and P. Rodriguez Cortes, "Opf-hgnn: Generalizable heterogeneous graph neural networks for ac optimal power flow," in *2024 IEEE Power & Energy Society General Meeting (PESGM)*, 2024, pp. 1–5.

[3] S. Ghamizi, A. Bojchevski, A. Ma, and J. Cao, "Safepowergraph: Safety-aware evaluation of graph neural networks for transmission power grids," *arXiv preprint arXiv:2407.12421*, 2024.

[4] Z. Yan and Y. Xu, "Real-time optimal power flow with linguistic stipulations: integrating gpt-agent and deep reinforcement learning," *IEEE Transactions on Power Systems*, 2023.

[5] C. Huang, S. Li, R. Liu, H. Wang, and Y. Chen, "Large foundation models for power systems," in *2024 IEEE Power & Energy Society General Meeting (PESGM)*. IEEE, 2024, pp. 1–5.

[6] A. G. et al., "The llama 3 herd of models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[7] T. B. B. et al., "Language models are few-shot learners," 2020. [Online]. Available: https://arxiv.org/abs/2005.14165

[8] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," 2021. [Online]. Available: https://arxiv.org/abs/2106.09685